CLA-3 – Case Study

Student template

Case study title	Used Car Price Prediction Based on Vehicle Age, Mileage and Brand
Base paper	Jha et al., “Used Car Price Prediction,” 2021
Date of submission	08-11-2026

Roll No.	**Name**
AP24110011946	Jaswanth Badeti

1. Introduction and Problem Statement

Used-car prices are difficult to estimate because a vehicle’s resale value depends on several interacting factors. Two cars of the same model may have different prices because of their manufacturing year, distance travelled, fuel type, transmission, ownership history and brand. An unrealistic asking price can disadvantage either the buyer or seller. A data-driven estimate can provide a useful reference point when evaluating a used vehicle.

This case study investigates the question: how effectively can supervised regression models estimate the selling price of a used car from its age, mileage and other available listing attributes? Three models are compared—Linear Regression, Decision Tree Regression and Random Forest Regression—using the same cleaned dataset and train-test split. Performance is evaluated using MAE, MSE, RMSE and the coefficient of determination (R^2).

2. Base Paper Summary

The base paper selected is “Used Car Price Prediction” by Abhishek Jha, Dr. Ramveer Singh, Manish, Imran Saifi and Shipra Srivastava, published in 2021 in the International Journal of Advance Research, Ideas and Innovations in Technology. The paper addresses the difficulty buyers and sellers face when judging whether an online used-car asking price is reasonable. It describes using a dataset collected from CarDekho and a machine-learning approach based on Random Forest Regression to estimate resale prices. Its described implementation uses Python and supporting data-science tools.

The paper’s central idea is that historical vehicle listings contain patterns linking vehicle attributes to selling price, and a trained regression model can learn these relationships. The paper’s accessible abstract identifies Random Forest Regression as its prediction algorithm, but does not provide a numerical benchmark that can be directly compared with this experiment. The present case study follows the same broad problem and dataset context, while extending the experiment by comparing three regression algorithms under one consistent evaluation procedure [1].

3. Method / Model Explanation

The task is a supervised regression problem. Each training example contains input features X and a continuous target y, where y is the listed selling price in Indian rupees. The notebook uses two numerical features—manufacturing year and kilometres driven—and five categorical features—fuel, seller type, transmission, owner category and brand.

Linear Regression estimates a weighted relationship between the input features and price. In simplified form, its prediction is $\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$. Here, $\hat{y}$ is the predicted price, β_0 is the intercept, β_i is the learned coefficient for feature i, and x_i is the corresponding processed feature value. It is simple and interpretable, but a straight-line relationship may not capture complex interactions in vehicle pricing.

A Decision Tree Regressor repeatedly divides the feature space into groups using feature-based conditions. Each final leaf predicts a value based on the training prices in that region. It can represent non-linear patterns, although a deep tree may fit noise. This experiment limits the tree depth to 12.

A Random Forest Regressor combines many decision trees. For regression, the forest prediction is the average of the individual tree predictions: $\hat{y} = (1/T)\,\Sigma_{t=1 \text{ to } T}\, h_t(x)$. Here, T is the number of trees, x is the input vehicle, and $h_t(x)$ is the prediction from tree t. Averaging helps reduce the variance of an individual tree. The experiment uses 250 trees, maximum depth 20 and minimum leaf size 2.

Before training, numerical features are standardised and categorical features are one-hot encoded. Standardisation transforms a

numerical value x into $z = (x - \mu)/\sigma$, where μ is the training mean and σ is the training standard deviation. One-hot encoding represents categories as binary indicator columns. The transformations are fitted within a pipeline using the training data.

Model quality is measured with four metrics. $MAE = (1/n)\Sigma|y_i - \hat{y}_i|$ gives the average absolute error in rupees. $MSE = (1/n)\Sigma(y_i - \hat{y}_i)^2$ squares errors and penalises large mistakes more heavily. $RMSE = \sqrt{MSE}$ returns the error to the original price unit. $R^2 = 1 - [\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2]$ measures the proportion of target variation explained relative to predicting the test-set mean. Here n is the number of test examples, y_i is the actual price, $\hat{y}_i$ is the predicted price and $\bar{y}$ is the mean actual price.

Worked example: the notebook's sample input is a 2018 Maruti car with 45,000 km driven, Petrol fuel, Individual seller, Manual transmission and First Owner status. The selected Random Forest pipeline returns an estimated selling price of 5,13,014.81. This is a ₹ model estimate based on learned patterns, not a verified current market quotation.

4. Implementation

The implementation was carried out in Google Colab using Python. The main libraries were pandas and NumPy for data handling, Matplotlib and Seaborn for visualisation, and scikit-learn for preprocessing, model training and evaluation. The dataset is the "CAR DETAILS FROM CAR DEKHO" CSV hosted in the bagassenop/cardekho GitHub repository [2]. It initially contained 4,340 records and eight columns: name, year, selling_price, km_driven, fuel, seller_type, transmission and owner.

The vehicle brand was extracted as the first word of the name field. Duplicate rows were removed; 763 duplicates were identified, leaving 3,577 records. The original dataset contained no missing values. The target was selling_price. The input features were year, km_driven, fuel, seller_type, transmission, owner and extracted brand.

The data was divided into 80% training data and 20% test data using random_state=42. This produced 2,861 training examples and 716 test examples. A ColumnTransformer standardised the two numerical features and one-hot encoded the five categorical features with handle_unknown='ignore'. Each model was placed in a scikit-learn Pipeline so that preprocessing and prediction were performed consistently.

The models were configured as follows: LinearRegression with default settings; DecisionTreeRegressor with max_depth=12 and random_state=42; and RandomForestRegressor with n_estimators=250, max_depth=20, min_samples_leaf=2, random_state=42 and n_jobs=-1. All models were trained on the same training split and evaluated on the same test split. The notebook also generated exploratory plots, an R^2 comparison chart and an actual-versus-predicted plot. The code and executed outputs are provided in the accompanying Google Colab notebook. Categorical attributes were handled using one-hot encoding, and duplicate listings were removed before splitting.

5. Results and Discussion

The test-set results are summarised in Table 1. Lower MAE, MSE and RMSE indicate smaller prediction errors, while a higher R^2 indicates a better fit relative to the test-set mean.

The experiment shows that Random Forest performed best among the three tested models. It achieved the lowest MAE (1,56,135.26) ₹ and RMSE (3,59,042.82), and the highest R^2 (0.600). Its R^2 indicates that the model explains approximately 60.0% of the variation in ₹ selling prices in this test set. Linear Regression ranked second with R^2 = 0.541, while the Decision Tree achieved R^2 = 0.331.

The closest test-set prediction had an actual price of 5,00,000 and a predicted price of 5,00,610.63, an absolute error of 610.63. This ₹ ₹ ₹ is a close prediction, but it should not be treated as proof that the model is generally accurate. The largest-error example had an actual price of 81,50,000 and a predicted price of 24,57,880, an absolute error of 56,92,120. The model substantially underpredicted this ₹ ₹ ₹ high-priced vehicle. This illustrates that performance can vary considerably across the price range and that expensive or unusual vehicles may be difficult for the model to estimate.

The results are limited by the dataset's size and coverage, the use of a single random train-test split, and the available attributes. The dataset covers vehicle years from 1992 to 2020 and reflects historical listings, so predictions should not be interpreted as current 2026 market valuations. Brand extraction using only the first word of the vehicle name is also a simplification. Additional features such as exact model/variant, location, condition, accident history and service records could improve predictions. The large error also suggests that performance should be evaluated separately across price ranges before practical use.

Model	MAE ( )₹	MSE	RMSE ( )₹	R^2
Linear Regression	179,642.89	1.480115×10^{11}	384,722.69	0.541
Decision Tree	184,795.22	2.153746×10^{11}	464,084.70	0.331
Random Forest	156,135.26	1.289117×10^{11}	359,042.82	0.600

6. Conclusion

This case study compared Linear Regression, Decision Tree Regression and Random Forest Regression for used-car price estimation. On the 716-example test set, Random Forest performed best, with MAE of 1,56,135.26, RMSE of 3,59,042.82 and R^2 of 0.600. The ₹ ₹ model can provide a useful approximate estimate from the available vehicle attributes, but the large error on one high-priced car shows that it is not reliable for every listing. The result answers the study question positively but with limitations: regression models can learn useful resale-price patterns, while broader data and richer vehicle information are needed for more dependable estimates.

7. References

[1] A. Jha, R. Singh, M. Manish, I. Saifi, and S. Srivastava, "Used car price prediction," International Journal of Advance Research, Ideas and Innovations in Technology, vol. 7, no. 4, 2021. [Online]. Available: https://www.ijariit.com/manuscript/used-car-price-prediction/

[2] bagassenop, "CAR DETAILS FROM CAR DEKHO.csv," GitHub repository. [Online]. Available: https://github.com/bagassenop/cardekho/blob/main/CAR%20DETAILS%20FROM%20CAR%20DEKHO.csv. Accessed: Nov. 8, 2026.

[3] scikit-learn developers, "scikit-learn: Machine Learning in Python," documentation. [Online]. Available: https://scikit-learn.org/stable/. Accessed: Nov. 8, 2026.

8. Declaration

I confirm that this case study is my own work and that all results reported were obtained by me.

Signature: _____

Roll No.: AP24110011946

Name: Jaswanth Badeti

Table 1. Regression model performance on the held-out test set. MSE is expressed in rupees squared.